

Intelligent Agents II

Lecture 3

CS 4880

Kirk Duran

Artificial Intelligence And Human Cognition

Key idea

AI mirrors human abilities: perception, memory, planning, and learning.

- ▶ AI aims to reproduce useful forms of intelligence and adaptability in machines.
- ▶ Definitions, models, and techniques in AI often reflect how we think humans reason and learn.
- ▶ The goal is not necessarily to copy biology exactly; it is to build systems that exhibit useful intelligent behavior.
- ▶ You already have an intuitive sense of many concepts that we will now formalize.

Review: Intelligent Agents

Definition

An **agent** perceives an environment through sensors and acts upon that environment through actuators.

Environment $\rightarrow$ Percepts $\rightarrow$ Agent $\rightarrow$ Actions $\rightarrow$ Environment

- ▶ A percept is what the agent senses at one moment.
- ▶ A percept sequence is the complete history of what it has sensed.
- ▶ The **agent function** maps percept histories to actions: $f : P^* \rightarrow A$.
- ▶ A rational agent chooses actions expected to maximize its performance measure given the information available.

Part 1: From Agent Functions To Agent Programs



The Table-Driven Approach

Key idea

Imagine a perfect lookup table containing the correct action for every possible percept sequence.

- ▶ Append the newest percept to the percept history.
- ▶ Use the complete percept sequence as a table key.
- ▶ Return the action stored in that entry.
- ▶ In principle, the agent needs no run-time reasoning or learning.

```
function TABLE-DRIVEN-AGENT(percept):  
    persistent percepts = []  
    persistent table  
  
    append percept to percepts  
    action = LOOKUP(percepts, table)  
    return action
```

Reality Check: Why A Table Does Not Work

Storage Explosion

The number of possible percept histories grows combinatorially or exponentially. Rich sensors make a realistic table astronomically large.

Impossible To Construct

Someone would need to precompute the correct action for every situation the agent could ever encounter.

No Generalization

A static table treats similar situations as unrelated entries instead of exploiting common structure.

No Room For Learning

New situations require new table entries; the representation itself provides no mechanism for adaptation.

Key idea

We need compact programs that reason from structure rather than enumerate every possible history.

Kinds Of Agent Programs

1. **Simple reflex agents:** react to the current percept.
2. **Model-based reflex agents:** maintain internal state.
3. **Goal-based agents:** reason about desired future states.
4. **Utility-based agents:** compare how desirable possible outcomes are.
5. **Learning agents:** improve behavior from experience.

Key idea

These are increasingly expressive design patterns, not mutually exclusive product categories. Real systems often combine them.

Part 2: Simple Reflex Agents

Is This Intelligence?

Image placeholder: stove-reflex.jpg

In class

A sensor detects a condition and immediately triggers a response. Is that intelligent behavior?

- ▶ It senses.
- ▶ It acts.
- ▶ It may satisfy an objective.
- ▶ It may not remember, plan, or learn.

Is This Intelligent?

Image placeholder: `water-fountain.jpg`

In class

An automatic bottle filler detects a bottle and turns on the water. What would make this an agent? What would make it intelligent?

Key idea

Intelligence is not necessarily all-or-nothing. Useful agent behavior can emerge from extremely simple decision rules.

Simple Reflex Agent

Definition

A **simple reflex agent** chooses an action using only the current percept.

Characteristics

- ▶ Ignores percept history.
- ▶ Maintains no internal world state.
- ▶ Uses **condition–action rules**.

Example

IF obstacle ahead **THEN** turn right.

Flow

1. Perceive the environment.
2. Match the percept to a rule condition.
3. Execute the associated action.
4. Repeat with no memory of the previous decision.

Simple Reflex Agents: Strengths And Weaknesses

Advantages

- ▶ Fast and computationally cheap.
- ▶ Requires little or no memory.
- ▶ Easy to implement and debug.
- ▶ Works well when the environment is fully observable and the needed information is available now.

Downside

- ▶ Missing information can cause incorrect actions.
- ▶ The same percept may require different actions in different hidden states.
- ▶ Local rules can create brittle or repetitive behavior.

Image placeholder: `simple-reflex-diagram.jpg`

Simple Reflex Example: Thermostat

Percept

Current room temperature.

Image placeholder: `thermostat-visual.jpg`

Condition–Action Rules

- ▶ If temperature is below the lower threshold, turn on heat.
- ▶ If temperature is within the desired band, maintain the current state.
- ▶ If temperature is above the upper threshold, turn off heat.

Key idea

The action depends on the current temperature, not a detailed history of the room.

Historical Reflex Example: Jacquard Loom

- ▶ Joseph-Marie Jacquard's early nineteenth-century loom used punched cards to control weaving patterns.
- ▶ A hole or no-hole pattern directly determined which threads were lifted.
- ▶ The loom did not represent the textile's purpose or "understand" the pattern.
- ▶ Its behavior was a direct response to the current card pattern.

Image placeholder: jacquard-loom.jpg

Example

Hole present → lift thread.

No hole → do nothing.

Key idea

A useful system can implement precise conditional behavior without a rich internal model.

Part 3: Model-Based Reflex Agents

Model-Based Reflex Agent

Definition

A **model-based reflex agent** maintains an internal state so it can reason about information that is not directly observable now.

Transition Model

Describes how the world changes over time and how the agent's actions change it.

Example

Turning the steering wheel changes a vehicle's heading.

Sensor Model

Describes how states of the world produce percepts.

Example

Rain on a camera lens can distort the observed image.

Key idea

Previous state + previous action + current percept → inferred current state → condition-action rule.

Model-Based Reflex Example: Self-Driving Car

- ▶ Track nearby vehicles even when they are temporarily occluded.
- ▶ Predict how those vehicles are likely to move.
- ▶ Estimate position, speed, lane, and surrounding road geometry.
- ▶ Base decisions on maintained state rather than one sensor frame alone.

Image placeholder: model-based-diagram.jpg

Downside

More memory and computation are required, and the agent can still make mistakes when its model is inaccurate or incomplete.

Reflex Or Model-Based Reflex?

Image placeholder: vacuum-reflex-or-model.jpg

In class

A robot vacuum maps rooms, remembers cleaned areas, and tracks its location. Is it still a simple reflex agent?

Look for:

- ▶ an internal map,
- ▶ memory of earlier percepts,
- ▶ prediction of hidden state,
- ▶ rules operating on inferred state.

Part 4: Goal-Based And Utility-Based Agents

Goal-Based Agent

Definition

A **goal-based agent** evaluates actions by whether their consequences move the world toward a desired goal state.

- ▶ Uses a model of how actions change the world.
- ▶ Adds explicit information about which outcomes are desirable.
- ▶ Can reason about the future instead of reacting immediately.
- ▶ Often uses search or planning to find a sequence of actions.

Flow

Perceive → update state → consult goal
→ search / plan → act → re-plan as
needed.

Goal-Based Architecture

Image placeholder: goal-based-diagram.jpg

- ▶ Goals describe what success looks like.
- ▶ The model predicts the consequences of actions.
- ▶ Search or planning finds a path from the current state to a goal state.

Key idea

A goal-based agent can choose actions that are useful only because of where they lead later.

Goal-Based Example: Vacuum Robot

Image placeholder: goal-vacuum.jpg

- ▶ Perceive dirt, obstacles, and current position.
- ▶ Update an internal map from previous movements and percepts.
- ▶ Check whether the goal is achieved: is the required area clean?
- ▶ If not, select an action or path that moves closer to the goal.
- ▶ Execute and repeat until the goal is met.

Is A Goal Enough?

Image placeholder: chess-goal.jpg

In class

A chess agent wants to win. Is “win the game” enough information to choose the next move?

- ▶ Win quickly or slowly?
- ▶ Keep more valuable pieces?
- ▶ Avoid risk?
- ▶ Prefer one winning line over another?

Key idea

Goals often give a binary answer: acceptable or not. They do not necessarily rank alternatives.

Utility-Based Example: Chess

Key idea

Even when the ultimate goal is “win,” an agent can evaluate intermediate states by estimating how desirable they are.

Possible evaluation features include:

- ▶ preserve valuable pieces,
- ▶ control the center,
- ▶ protect the king,
- ▶ avoid weak pawn structures,
- ▶ maintain mobility,
- ▶ increase the probability of eventually winning.

Example

A utility value compresses several strategic preferences into a quantity the agent can compare.

Utility-Based Agent

Definition

A **utility function** assigns a numerical value to states or outcomes representing how desirable they are to the agent.

- ▶ Ranks alternatives instead of treating success as yes/no.
- ▶ Handles conflicting objectives.
- ▶ Under uncertainty, compares actions by **expected utility**.

Image placeholder: utility-based-diagram.jpg

$$EU(a) = \sum_s P(s \mid a, e)U(s)$$

Utility-Based Example: Self-Driving Taxi

A rational taxi may optimize several objectives simultaneously:

- ▶ shortest travel time,
- ▶ least fuel or energy consumption,
- ▶ lowest accident risk,
- ▶ passenger comfort,
- ▶ compliance with traffic laws.

Why Utility Helps

It supports tradeoffs such as speed versus safety and allows the agent to compare several acceptable outcomes.

Tradeoff

Utility-based reasoning can be computationally expensive because the agent may need to evaluate many possibilities and uncertain outcomes.

Goals Or Utility Are Not Enough

Image placeholder: deepblue-alphago.jpg

- ▶ Deep Blue used massive search plus carefully engineered evaluation methods to defeat Garry Kasparov.
- ▶ AlphaGo combined search with learned policy and value models to defeat Lee Sedol in 2016.
- ▶ A goal or utility function tells an agent what it wants; it does not automatically tell the agent how to act well.

Key idea

Intelligent systems often combine representation, models, search, utility, and learning.

Part 5: Learning Agents

Learning Agents

Definition

A **learning agent** improves its behavior over time based on experience or feedback.

1. Perceive the environment.
2. Take an action using the current agent program.
3. Observe the outcome.
4. Use feedback to improve internal models, rules, utilities, or strategies.
5. Repeat with better knowledge next time.

Key idea

Learning allows an agent to begin with incomplete knowledge, adapt to an unfamiliar environment, and improve through experience.

Four Components Of A Learning Agent

Performance Element

Chooses the actions that interact with the external world.

Learning Element

Modifies the performance element to improve behavior from experience.

Critic

Evaluates how well the agent is performing relative to the performance measure.

Problem Generator

Suggests exploratory actions that may produce informative experience.

Key idea

Learning separates acting, evaluating, improving, and exploring.

Learning Agent Example: Self-Driving Car

- ▶ **Performance element:** chooses braking, acceleration, and steering actions.
- ▶ **Critic:** evaluates whether braking was too hard, too soft, unsafe, or uncomfortable.
- ▶ **Learning element:** updates a transition model, control policy, or prediction model based on feedback.
- ▶ **Problem generator:** encourages informative experience, such as varied braking maneuvers under safe conditions.

Example

Observed outcome → feedback → model or policy update → improved future action.

Part 6: Problem Formulation And Abstraction

Problem Abstraction

Definition

Abstraction represents a real problem using only the details needed for reasoning about a solution.

- ▶ Real road travel includes weather, traffic, passengers, music, laws, road paint, and countless other facts.
- ▶ A navigation model may keep only locations, roads, travel costs, and constraints.
- ▶ Cities or intersections become **states**.
- ▶ Roads become **actions or transitions**.
- ▶ Irrelevant details are filtered out.

Image placeholder:

problem-abstraction-navigation.jpg

Formal Search Problem Formulation

A search problem is commonly described by:

- ▶ **Initial state:** where the agent starts.
- ▶ **Actions:** the choices available in a state.
- ▶ **Transition model:** the state produced by taking an action.
- ▶ **Goal test:** whether a state satisfies the objective.
- ▶ **Path cost:** the accumulated cost of a sequence of actions.

Key idea

Search algorithms operate on this abstract state space rather than on every detail of the physical world.

Part 7: Problem Types

Single-State Problem

Definition

A **single-state problem** assumes the agent knows exactly which state it is in and can predict the result of each action.

Typical assumptions:

- ▶ the environment is fully observable,
- ▶ the transition model is known,
- ▶ action outcomes are deterministic.

Key idea

The agent can plan a complete sequence of actions without uncertainty about its current state or the immediate effect of an action.

Chess As A Single-State Problem

Image placeholder: chess-board.jpg

- ▶ The current board position is fully visible.
- ▶ Legal moves are known.
- ▶ Executing a legal move has a deterministic result.
- ▶ The agent therefore knows exactly what board state each move will produce.

Key idea

Chess is still strategically difficult because another agent chooses future moves; “single-state” refers to state certainty, not overall simplicity.

Multiple-State Problem

Definition

In a **multiple-state problem**, the agent cannot identify one exact current world state from its percepts.

- ▶ The world may be partially observable.
- ▶ Several physical states may be consistent with the same percept history.
- ▶ The agent therefore reasons about a set of possible current states.

Example

A robot with limited sensors may know that it is in one of several rooms without knowing exactly which room.

Belief States

Image placeholder: multiple-state-map.jpg

Definition

A **belief state** represents all states the agent currently considers possible.

- ▶ New percepts eliminate inconsistent possibilities.
- ▶ Actions transform the set of possible states.
- ▶ Planning occurs over belief states when the true state is hidden.

Key idea

The agent reasons about what it *believes* the world could be, not only one assumed world state.

Contingency Problems

Definition

A **contingency problem** requires a plan that can react to different observations or outcomes encountered during execution.

- ▶ The environment may be nondeterministic: the same action can lead to different results.
- ▶ External agents or physical uncertainty may interfere with the expected result.
- ▶ The agent may not have complete control over the environment.
- ▶ New observations can force the plan to branch.

Example

Drive toward the destination. If the highway is clear, continue. If an accident blocks it, use the alternate route. If both are blocked, stop and re-plan.

Exploration Problems

Definition

In an **exploration problem**, the agent does not initially know enough about the environment to plan completely in advance.

- ▶ The agent may not know the full state space, available actions, or transition model.
- ▶ It gathers information by interacting with the environment.
- ▶ It must build or refine a model while simultaneously attempting to achieve its goal.
- ▶ Exploration trades immediate performance for information that may improve future decisions.

Key idea

The agent learns how the world works by acting in it.

Exploration Example: Self-Driving Car

- ▶ The car enters an unmapped city or road network.
- ▶ It does not begin with complete knowledge of the roads.
- ▶ It explores streets and intersections while localizing itself.
- ▶ It incrementally builds a map and improves its routing.

Image placeholder: `exploration-car.jpg`

Example

Simultaneous localization and mapping (SLAM) is a classic example of building a model while navigating.

Problem Domains

Puzzles And Mazes

- ▶ Sliding-tile puzzle such as the 8-puzzle.
- ▶ A maze with full observability.
- ▶ A maze where the agent does not know its starting location.
- ▶ A partially completed jigsaw puzzle without edge pieces.

Strategy And Simulation

- ▶ Minesweeper.
- ▶ Battleship.
- ▶ StarCraft.
- ▶ Escape rooms and other interactive environments.

In class

For each problem, identify what the agent knows, what it does not know, and whether outcomes are deterministic.

Part 8: Putting The Architectures Together

Agent Architectures: What Changes?

Architecture	Uses	Main Question	Key Addition
Simple reflex	Current percept	What should I do now?	Condition-action rules
Model-based reflex	Percept + internal state	What is the world like now?	Memory and model
Goal-based	State + goals	How can I reach success?	Search / planning
Utility-based	State + utilities	Which outcome is best?	Preferences and tradeoffs
Learning	Experience + feedback	How can I improve?	Adaptation over time

A Modern Agent Is Usually Hybrid

Key idea

Real AI systems commonly combine several of today's architectures rather than choosing exactly one.

A self-driving car may:

- ▶ use reflexes for emergency braking,
- ▶ maintain a model of surrounding traffic,
- ▶ plan toward a destination,
- ▶ optimize safety, time, energy, and comfort,
- ▶ learn from data and experience.

In class

Given an environment and a performance measure, what internal structure does the agent actually need?

Next Time: Search Algorithms

Image placeholder: `search-maze.jpg`

Key idea

Goal-based agents need a systematic way to find sequences of actions that move from an initial state to a goal state.

- ▶ State spaces.
- ▶ Search trees.
- ▶ Breadth-first and depth-first search.
- ▶ Cost and optimality.
- ▶ Heuristics later.